

南開大學
Nankai University

南　　开　　大　　学

计　算　机　学　院

编译原理上机实验报告

预备工作：了解你的编译器、LLVM IR 与汇编编程

姓名：张龙飞、袁煜杰

学号：2413625、2412303

专业：计算机科学与技术

2026 年 10 月 11 日

摘要

本实验通过编译工具的阶段输出研究语言处理流程，并设计六组 SysY 示例，编写等价的 LLVM IR 与 RISC-V 汇编，涵盖算术、分支、循环、函数、数组和浮点转换。通过参考程序、本机及 RV64 运行对照，共 269 项检查全部通过。进阶部分结合 AscendNPU IR 官方 VecAdd 文档分析方言层级，并执行上游 MLIR 从 Linalg 经 SCF、CF、LLVM 方言到 LLVM IR 的降低，CPU VecAdd 的 16 个输出元素均正确。

关键词：SysY；LLVM IR；RISC-V；SSA；MLIR；等价性验证

目录

一、 引言 **1**
（一） 实验背景与目的 1
（二） 工作流程实验环境 1
（三） IR 与 MLIR 实验环境 1

二、 基础要求 **1**
（一） 了解编译器工作过程 1
（二） 熟悉 LLVM IR 中间语言 4
（三） 熟悉汇编语言 6

三、 进阶要求 **8**
（一） 观察 AI 编译器 MLIR 中端处理过程 8

四、 总结与体会 **10**

一、 引言

（一） 实验背景与目的

SysY 是全国大学生编译系统设计赛定义的简化 C 语言。本课程上机大作业要求实现一个 SysY 编译器（词法分析、语法分析、类型检查与中间代码生成、代码优化、目标代码生成），目标平台为 RISC-V。本次预备工作是上机大作业的第一个模块，目的在于：

1. 以 GCC / LLVM(Clang) 为研究对象，深入探究语言处理系统的完整工作过程，理解预处理器、编译器（前端与后端各阶段）、汇编器、链接器的功能；
2. 熟悉 LLVM IR 中间语言与 RISC-V 汇编语言，设计覆盖 SysY 语言特性的示例程序，手写等价的 LLVM IR 与 RISC-V 汇编程序，链接 SysY 运行时库并验证运行结果的正确性；
3. 为后续词法分析器、语法分析器、中间代码生成与目标代码生成等模块的实现建立直观认识与技术储备。

（二） 工作流程实验环境

本实验在 WSL2（Windows Subsystem for Linux）中进行，使用 QEMU 用户态模拟器运行 RISC-V 目标程序。主要工具及版本如表 1 所示。

项目	版本 / 说明
操作系统	WSL2：Ubuntu 22.04.5 LTS（x86_64）
GCC / G++	11.4.0
Clang / LLVM 工具链	14.0.0（含 llvm-as、lli、llc、opt、llvm-dis）
Flex / Bison	2.6.4 / 3.8.2
RISC-V 交叉工具链	riscv64-linux-gnu-gcc / g++ 11.4.0
模拟器	QEMU 7.0.0（qemu-riscv64 用户态模式，/opt/qemu 源码编译）

表 1: 实验环境与工具版本

（三） IR 与 MLIR 实验环境

实际运行环境为 WSL Ubuntu 26.04.1 LTS（x86_64），Clang/LLVM 21.1.8，RISC-V GCC 15.2.0，QEMU 10.2.1。MLIR 21.1.8 工具从 Ubuntu 官方源下载并在项目内解压。详细命令和版本保存在实验结果目录中。

二、 基础要求

（一） 了解编译器工作过程

以阶乘程序 `fact.cpp` 为例，通过命令行选项获取各阶段产物（表 2）。

2.1.1 实验方案

研究载体：阶乘程序 fact.cpp（PPT 示例）

```
#include <iostream>
using namespace std;
int main()
{
    int i, n, f;

    cin >> n;
    i = 2;
    f = 1;
    while (i <= n)
    {
        f = f * i;
        i = i + 1;
    }
    cout << f << endl;
}
```

阶段	命令选项	产物
预处理	`clang++ -E`	`fact.i`
编译（前端）	`clang++ -emit-llvm -S`	`fact.ll`（LLVM IR）
编译（后端）	`g++ -S` / `riscv64-linux-gnu-g++ -S`	`fact.s`（汇编）
汇编	`clang++ -c`	`fact.o`（目标文件）
链接	`g++` / `riscv64-linux-gnu-g++ -static`	可执行程序
运行验证	`lli` / `qemu-riscv64`	运行结果

表 2: 各编译阶段对应的命令选项与产物

2.1.2 预处理器

源程序仅 17 行，经 `clang++ -E` 预处理后得到 30194 行的 `fact.i`——C++ 标准库头文件 `<iostream>` 展开了数万行声明。

2.1.3 编译器

编译前端：生成 LLVM IR `clang++ -emit-llvm -S` 生成 111 行 LLVM IR（`fact.ll`），关键片段如下：

fact.ll 关键片段

```
@_ZSt3cin  = external global %"class.std::basic_istream", align 8
@_ZSt4cout = external global %"class.std::basic_ostream", align 8

define dso_local noundef i32 @main() #4 {
  %1 = alloca i32, align 4      ; return value
  %2 = alloca i32, align 4      ; i
  %3 = alloca i32, align 4      ; n
  %4 = alloca i32, align 4      ; f
```

```
  store i32 0, i32* %1, align 4
  %5 = call ... @_ZNSirsERi(@_ZSt3cin, i32* %3)      ; cin >> n
  store i32 2, i32* %2, align 4                      ; i = 2
  store i32 1, i32* %4, align 4                      ; f = 1
  br label %6
6:                                                   ; while (i <= n)
  %7 = load i32, i32* %2, align 4
  %8 = load i32, i32* %3, align 4
  %9 = icmp sle i32 %7, %8
  br i1 %9, label %10, label %16
10:
  %11 = load i32, i32* %4
  %12 = load i32, i32* %2
  %13 = mul nsw i32 %11, %12                         ; f = f * i
  ...
16:
  %17 = load i32, i32* %4, align 4
  %18 = call ... @_ZNSolsEi(@_ZSt4cout, i32 %17)     ; cout << f
  %19 = call ... @_ZNSolsEPFRSoS_E(%18, ... endl)    ; << endl
  ret i32 %20
}
```

编译后端：生成目标汇编 分别用 `g++ -S`（x86_64，AT&T 风格）与 `riscv64-linux-gnu-g++ -S`（RISC-V）生成汇编，对比两者风格。

2.1.4 汇编器

`clang++ -c` 将汇编翻译为目标文件 `fact.o`，可用 `objdump -d` 反汇编查看机器码，`objdump -r` 查看重定位项。

2.1.5 链接器

链接阶段将目标文件与 C++ 运行库（`std::cin`/`std::cout` 所在的 libstdc++）组装为可执行程序。

2.1.6 运行验证

四种方式运行同一程序，结果一致（输入 5，输出 120），说明各阶段产物等价：

运行验证

```
$ echo 5 | ./fact_x86                # x86 可执行（g++ 链接产物）
120
$ echo 5 | lli fact.ll               # LLVM IR 解释执行
120
$ echo 5 | ./fact_fromll_x86         # 由 IR 直接重编译的可执行
120
$ echo 5 | qemu-riscv64 ./fact_rv64  # RISC-V 可执行（QEMU 模拟）
120
```

2.1.7 进一步探索：优化选项的影响

调整优化级别（`-O0/-O1/-O2/-Os`），观察 LLVM IR 与 RISC-V 汇编的变化，结果如表 3 所示。

程序 / 产物	-O0	-O1	-O2	-Os
fact：LLVM IR 行数	111	163	250	152
fact：RISC-V 汇编行数	156	97	146	89
fib：LLVM IR 行数	124	248	247	153
fib：RISC-V 汇编行数	177	156	231	118

表 3: 不同优化级别下 LLVM IR 与 RISC-V 汇编的规模对比

（二） 熟悉 LLVM IR 中间语言

2.2.1 实验方法

设计六组 SysY 程序，分别编写 LLVM IR 与 RISC-V 汇编。整数和浮点数分别用 i32、float 表示；局部寄存器遵循 SSA，控制流合流使用 phi，内存变量通过 load/store 访问 [2]。

2.2.2 编译与验证流程

使用 Clang/LLVM 检查、优化并编译 IR，链接 getint、putint、putch 完成输入输出。验证脚本比较参考程序、手写 IR 和手写汇编的标准输出及退出码。

```
python3 prep/llvm_ir/verify.py
llvm-as sample.ll -o sample.bc
opt -passes=verify -disable-output sample.ll
opt -S '-passes=default<O2>' sample.ll -o sample_O2.ll
llc -mtriple=riscv64-linux-gnu -mattr=+m,+a,+f,+d,+c \
    -target-abi=lp64d sample.ll -o sample_generated_rv.s
```

2.2.3 示例设计与特性覆盖

示例	主要覆盖	输入数
arithmetic	加减乘除与余数、单目正负、常量、块遮蔽、逻辑非转整数	6
short_circuit	if/else、逻辑与或短路、全局副作用、自定义函数	6
loop_control	while、continue、break、合流与多条回边	6
functions	带参函数、递归、整数返回、void 函数及返回	5
array_float	二维数组、部分初始化、数组形参、浮点运算与转换	5
promote_demo	内存形式循环及 mem2reg 对照	5

表 4: 本次手写 LLVM IR 的实验设计

算术例子输入 n，计算 $(n+3)\times 2-n/2+n\%3+7$，并分别输出 $-n$、$+n$、n+!n。内层同名变量固定为 7，外层 n 在后续输出中仍保持原值，用于验证遮蔽而非覆盖。负数整除按向零截断计算，余数为 $n-\mathrm{trunc}(n/3)\times 3$。逻辑非先由 icmp 产生 i1，再由 zext 转为 i32 后参与加法。

2.2.4 短路控制流与可观测副作用

短路例子包含 `n != 0 && 12 / n > 2` 和 `n == 0 || touch()`。touch 每次调用将全局 hits 加一。以 n=0 为例，正确程序既不除以零，也不调用 touch，最终 hits=0；非零输入则调用 touch 一次。

```
%nonzero = icmp ne i32 %n, 0
br i1 %nonzero, label %and.rhs, label %and.false
and.rhs:
  %q = sdiv i32 12, %n
  %large = icmp sgt i32 %q, 2
  br i1 %large, label %and.true, label %and.false
and.join:
  %and.value = phi i32 [1, %and.true], [0, %and.false]
```

右操作数置于独立基本块中，由分支决定是否执行。测试 n=-3、0、1、4、5、12，覆盖受保护分母、条件真/假和副作用次数。

2.2.5 循环、函数与数据表示

循环例子逐个处理 1 到 n，跳过 2，在数值超过 5 时退出。continue 的边直接回到循环头，break 的边直接进入出口。循环头分别对 i 和 sum 建立 phi，continue 边保持 sum，普通边传入累加后的值。出口的 phi 对正常条件退出和 break 退出分别提供值。

```
header:
  %i = phi i32 [0, %entry], [%next, %skip], [%next, %accumulate]
  %sum = phi i32 [0, %entry], [%sum, %skip], [%added, %accumulate]
  %more = icmp slt i32 %i, %n
  br i1 %more, label %body, label %done
done:
  %result = phi i32 [%sum, %header], [%sum, %break.test]
```

LLVM verifier 检查类型、前驱和支配关系，运行测试检查输出。输入 n=2 时只累加 1，n=5 或 n=8 时结果均为 13，分别触发 continue 与 break。函数例子进一步使用递归阶乘和 void 输出函数，检查参数、call 和 ret 的对应关系。

数组/浮点例子将 `int a[2][3]=1,2,3,4` 表示为二维数组对象，初始化第二行剩余元素为零。数组实参传地址，按 `[3 x i32]` 的行类型通过 getelementptr 寻址，两行和为 10。浮点过程为 $\mathrm{twice}(1.5n+0.5)=3n+1$，使用 sitofp、fmul、fadd、fptosi 和 fcmp，既验证 int/float 转换，也验证 float 参数和返回值。

2.2.6 mem2reg 观察与结果验证

另手写一份以 alloca/load/store 保存循环变量的求和程序，使用 LLVM 自带 `opt -passes=mem2reg` 得到可提升变量的寄存器形式。转换前为 2 个 alloca、3 个 load、4 个 store、0 个 phi；转换后这些局部访存操作均消失，形成 2 个 phi。对 n=-3、0、1、5、10 分别运行，转换前后输出一致。

（三） 熟悉汇编语言

2.3.1 手写 RISC-V 汇编的组织与调用约定

六组示例采用 RV64GC 指令集和 LP64D 调用约定。整数参数和返回值使用 a0 等参数寄存器，浮点函数的单精度参数和返回值使用 fa0。遵循 RISC-V psABI[1]，函数调用处保持 sp 的 16 字节对齐，使用 s 寄存器的函数保存并恢复相应值；非叶函数保存返回地址 ra。跨调用保留的数据存入 s 寄存器或栈中。

文件	实现重点
arithmetic_rv.s	word 算术、向零截断除法、有符号余数、跨输出调用保存输入
short_circuit_rv.s	受保护除法、分支短路、全局 hits 与 touch 副作用
loop_control_rv.s	continue 回边、break 出口、循环状态寄存器
functions_rv.s	递归栈帧、保存 ra 与原参数、void 输出函数
array_float_rv.s	二维数组行步长、补零、float ABI、显式截断转换
promote_demo_rv.s	寄存器求和循环，与内存形式 LLVM IR 对照

表 5: 手写汇编与同名 SysY、LLVM IR 示例的对应关系

SysY 的 int 是 32 位，而 RV64 的整数寄存器是 64 位。因此使用 addw、addiw、subw、mulw、divw、remw 等指令处理整数计算，lw 读取数组元素，并保持 32 位结果的符号扩展。寻址和 sp 调整则使用 64 位指令。伪指令 call、ret、li、la 由汇编器展开。

2.3.2 短路、循环与递归的目标代码表达

短路例子先检查 n，再进入右侧表达式。以下为受保护除法的完整控制片段：

```
    li a0,0
    beqz s0,.Land_done
    li t0,12
    divw t0,t0,s0
    li t1,2
    slt a0,t1,t0
.Land_done:
    call putint
```

其中 s0 保存 n。n=0 时直接跳过 divw；另一个逻辑或条件在 n=0 时直接进入真分支，不会调用 touch。全局 hits 通过 la/lw/sw 访问，输出次数与 LLVM IR 版本一致。循环例子用寄存器保存 i 与 sum，continue 跳回条件检查，break 跳到输出出口。LLVM IR 中的 phi 对应不同路径对状态寄存器的更新。

递归 factorial 的非基本情形保存原参数和 ra，再调用自身：

```
    addi sp,sp,-16
    sd ra,8(sp)
    sd s0,0(sp)
    mv s0,a0
    addiw a0,a0,-1
    call factorial
    mulw a0,s0,a0
    ld s0,0(sp)
    ld ra,8(sp)
    addi sp,sp,16
    ret
```

基本情形 n<=1 直接返回 1，不建立额外栈帧。emit 不返回计算值，但它调用输出库，也必须保存 ra；源程序的 void 返回对应恢复调用现场后 ret。测试 n=0、1、5、7、12，既检查递归出口也检查多层返回。

2.3.3 数组布局与浮点数实现

二维数组采用行优先布局，每行含三个 i32 元素，因此行步长为 12 字节。row_sum 接收 a0 中的数组首地址和 a1 中的行下标，用 $\text{base} + 12 \times \text{row} + 4 \times \text{column}$ 计算元素地址。main 的 64 字节栈帧中，偏移 0–20 保存六个元素，24 保存 scaled，40、48、56 分别保存 s1、s0、ra；未初始化的两个数组元素显式写零。输入和输出库调用使用独立栈帧，不覆盖调用者的数组区域。

整数输入通过 fcvt.s.w 转换为 float，fmul.s 与 fadd.s 计算 $1.5n + 0.5$，twice 在 fa0 中收参并返回 $3n + 1$。fcvt.w.s 的 rtz 模式显式实现 float 到 int 的向零截断。scaled 在后续 row_sum、putint、putch 调用期间保存在栈上，最后重新加载并用 flt.s 比较。

2.3.4 汇编、链接和同输入等价性验证

验证脚本对每个手写文件执行静态链接并由 QEMU 运行，例如：

```
riscv64-linux-gnu-gcc -static -march=rv64gc -mabi=lp64d \
  prep/llvm_ir/handwritten/functions_rv.s prep/manual/sylib.c \
  -o build/prep-llvm-ir/functions_handwritten_rv
printf '7\n' | qemu-riscv64 build/prep-llvm-ir/functions_handwritten_rv
# stdout: 5040, process exit status: 0
```

汇编器生成目标代码，链接器解析库函数，QEMU 执行 RV64 程序。

每个输入比较八条路径：C 参考程序本机/RV64、手写 IR 本机/RV64 （原始与 O2）、lli、手写 RV64 汇编。33 组输入共 264 项，加 mem2reg 的 5 项为 **269/269 通过**。arithmetic、short_circuit、loop_control 各 48 项，functions 与 array_float 各 40 项，promote_demo 含 mem2reg 共 45 项。全部输出逐字比较，并检查进程退出码为 0。

示例	代表输入	源程序/IR/手写汇编 stdout	验证重点
arithmetic	-7	1, 7, -7, -7	负数除法与余数
short_circuit	0 / 4	0, 2, 0 / 1, 2, 1	短路与副作用
loop_control	2 / 8	1 / 13	continue / break
functions	7	5040	递归与返回
array_float	-3 / 2	10, -8, 0 / 10, 7, 1	数组、浮点与转换
promote_demo	5	10	循环累加

表 6: 实际执行的代表结果，逗号表示换行分隔的输出

三、 进阶要求

（一） 观察 AI 编译器 MLIR 中端处理过程

3.1.1 研究方法

依据 AscendNPU IR 官方 VecAdd 示例和方言文档分析中端层级，并使用 MLIR 21.1.8 构造上游 VecAdd，执行逐层降低和 CPU 运行验证。

3.1.2 AscendNPU IR VecAdd 的方言与层级

官方快速入门 [3] 的 VecAdd 使用 i16 元素，并区分 GM 与 UB 地址空间。输入先搬入片上缓冲，再进行向量加法，最后搬回输出；设备入口用 HACC 属性表达。编译后还需要 CANN runtime 注册和启动设备程序。

架构说明 [4] 把 HFusion、HIVM、HACC 分为高层融合、硬件相关计算/搬运/同步、异构执行表达等职责。结合 HIVM 文档 [5]，可以看到片上内存和计算流水信息在特定层被显式保留。官方快速入门从 HIVM 级程序开始，体现片上缓冲、向量计算和数据搬运的配合。

比较点	上游 CPU 实验	Ascend 官方文档路线
高层计算	Linalg 通用逐元素计算	HFusion 等较高层表达
硬件相关操作	普通 MemRef 访存	HIVM 中的计算、搬运、同步
内存空间	本机普通内存	GM、UB 等硬件空间
执行接口	MLIR 生成 C 包装函数	HACC 入口及 CANN runtime
证据	本机各层输出与运行检查	官方示例和架构文档分析

表 7: 两种路线的职责与证据差异

3.1.3 可执行的上游 VecAdd 设计

独立设计的 `prep/mlir/vecadd.mlir` 使用三个 `memref<16xi16>` 参数。Linalg 的三个恒等索引映射表示 $c[i] = a[i] + b[i]$，迭代属性 parallel 表明各元素计算互相独立。本实验将其降低为标量循环。

```
linalg.generic {
  indexing_maps = [affine_map<(i)->(i)>,
                   affine_map<(i)->(i)>, affine_map<(i)->(i)>],
  iterator_types = ["parallel"]
} ins(%a, %b : memref<16xi16>, memref<16xi16>)
  outs(%c : memref<16xi16>) {
^bb0(%x: i16, %y: i16, %old: i16):
  %sum = arith.addi %x, %y : i16
  linalg.yield %sum : i16
}
```

输入数组设为 $a[i] = i$、$b[i] = 1$（$0 \le i < 16$），C 驱动逐项检查输出。

3.1.4 逐层 lowering 与检查点

上游 MLIR 文档介绍结构化 SCF 与非结构化 CF 的区分 [6]。本次根据工具可用 pass[7] 执行如下管线，每一步均保存文件：

```
mlir-opt vecadd.mlir -o 00_linalg.mlir
mlir-opt 00_linalg.mlir --convert-linalg-to-loops -o 01_scf.mlir
mlir-opt 01_scf.mlir --convert-scf-to-cf -o 02_cf.mlir
mlir-opt 02_cf.mlir --convert-to-llvm \
    --reconcile-unrealized-casts -o 03_llvm.mlir
mlir-translate --mlir-to-llvmir 03_llvm.mlir -o 04_llvm.ll
```

各阶段输出由验证脚本保存。

检查点	行数	实际观察到的操作方言
00_linalg	12	Linalg、Arith、Func
01_scf	15	SCF、MemRef、Arith、Func
02_cf	21	CF、MemRef、Arith、Func
03_llvm	67	LLVM

表 8: MLIR 降低阶段与操作方言

首先 generic 被展开为 `scf.for`，循环体出现两个 memref.load、一次 arith.addi 和一次 memref.store。接着循环被拆成比较、条件分支、循环体和回边，迭代变量通过块参数传递；结构化循环这个概念不再显式保留。再向 LLVM 方言转换时，MemRef 描述符中的已分配地址、对齐地址、偏移、长度和步长被显式展开，生成与 C 调用者衔接的包装函数。最后 `mlir-translate` 才输出真正的 LLVM IR。因此 `llvm.*` 操作所在的 MLIR 文件，与 LLVM 自身的`.ll` 文件并不是同一种文本格式。

3.1.5 末端验证与分析

C 驱动构造与一维 MemRef 对应的描述符，通过 `_mlir_ciface_vecadd` 调用降低后的程序。本次 LLVM verifier 检查通过，Clang 编译链接后实际输出：

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
```

驱动逐个断言全部 16 个元素正确，退出码 0。版本、命令、stderr 和结果保存于 `prep/mlir/results/`。行数从 12 增至 67，主要来自内存描述符与调用接口的展开。

多级 IR 在高层保留索引映射和计算结构，在中层明确循环与访存，最终转换为指针操作和调用接口。各阶段的类型、控制流和内存表示随降低同步转换。

四、 总结与体会

编译阶段输出展示了源程序向 IR、汇编、目标文件和可执行程序的转换。手写 LLVM IR 与 RISC-V 汇编进一步说明短路求值、循环合流、数组布局和调用约定的实现方式。六组程序在八条执行路径上输出一致，连同 mem2reg 对照，269 项检查全部通过。MLIR VecAdd 实验展示了从高层计算结构到循环、分支和指针操作的逐层降低，16 个输出元素全部正确。实验将语言语义、中间表示与目标执行结果建立了对应关系。

参考文献

[1] RISC-V International. RISC-V ABIs Specification. https://riscv-non-isa.github.io/riscv-elf-psabi-doc/.

[2] LLVM Project. LLVM Language Reference Manual. https://llvm.org/docs/LangRef.html.

[3] AscendNPU IR. 快速入门示例. https://ascendnpu-ir.gitcode.com/zh_cn/sources/introduction/quick_start/examples_zh.html.

[4] AscendNPU IR. 架构介绍. https://ascendnpu-ir.gitcode.com/zh_cn/sources/introduction/architecture_zh.html.

[5] AscendNPU IR. HIVM Dialect. https://ascendnpu-ir.gitcode.com/zh_cn/sources/developer_guide/dialects/HIVMDialect.html.

[6] MLIR Project. SCF Dialect. https://mlir.llvm.org/docs/Dialects/SCFDialect/.

[7] MLIR Project. Passes. https://mlir.llvm.org/docs/Passes/.